

RELAZIONE DI TANIA DENTE

Il seguente codice che ho sviluppato risponde ai quesiti A2, B3, e C1 della consegna. L'analisi è stata strutturata in modo lineare, processando i casi singolarmente.

Ho scelto di lavorare nell'ambiente Visual Studio Code e di aprire i file csv da questo per facilitare la visualizzazione dei miei dati e la gestione dei file intermedi che ho creato per isolare le informazioni utili per rispondere alle domande.

Una volta scaricati ed estratti i file contenuti nello zip, li ho aperti specificando la codifica `latin-1`, operazione per me necessaria dovuta agli errori di decodifica che mi si presentavano con la codifica standard `UTF-8`.

Inizialmente ho riportato le funzioni, tre delle quali denominate `riempimento_file` seguite dal nome del quesito di riferimento. Il loro scopo è quello di poter riportare, in un nuovo file, i dati necessari per rispondere alle domande, per poi lavorare su questi. La logica che ho seguito per queste funzioni è molto simile, motivo per il quale i loro nomi sono quasi identici. Come primo passaggio, ho eliminato la prima riga, utile per l'identificazione manuale delle colonne ma superflua per l'elaborazione automatizzata dei record.

Attraverso un ciclo `for` ogni riga, fino alla fine di queste, viene processata dalla funzione `divisione`, per poter generare dei confini e quindi per separare correttamente le informazioni. Una volta suddivisi i record, su ognuna delle tre funzioni ho inserito delle istruzioni condizionali specifiche. Nel quesito A2, la condizione riguarda la variabile `piattaforma_fissa`, che rappresenta il tipo di piattaforma sulla quale il gioco può essere letto. Nei file PS4 e XboxOne la piattaforma è implicita nel titolo, ma assente come colonna testuale. Essendo fisse, queste vengono specificate, mentre nel terzo file la piattaforma deve essere letta da una colonna specifica che, appunto, varia di gioco in gioco. Nelle domande B3 e C1 effettuo invece una pulizia dei dati, in cui elimino i record che riportano dati non significativi alla risoluzione finale. I dati filtrati e formattati vengono infine scritti su nuovi file di output, pronti per le analisi statistiche successive.

L'ultima funzione del mio codice è `divisione`, progettata per risolvere una problematica riscontrata più volte in questo percorso, ovvero la divisione dei dati ogni qual volta che si riscontrava una virgola ad eccezione dei campi che vengono racchiusi tra virgolette doppie.

Inizio generando due liste vuote. Nel caso in cui all'interno del mio record non ci sono virgolette, questo viene diviso da una funzione di `split` ogni qual volta che si presenta una virgola, restituendo una lista di stringhe separate dalla virgola. Nel caso di virgolette, la funzione implementa un algoritmo di scansione e ricostruzione. Tramite un primo ciclo `for`, la funzione inserisce nella lista `index_virg` gli indici di tutte le virgolette presenti. Con un secondo ciclo `for` vengono analizzate le coppie di virgolette e per ognuna estraggo la parte di testo precedente ad essa e quella contenuta nella coppia, eliminando eventualmente spazi vuoti. Concludo assemblando queste parti del mio record insieme, aggiungendo l'ultima parte della stringa, cioè il resto, per poi restituire la riga suddivisa correttamente.

Da questo punto in poi, inizio con la risoluzione del primo quesito.

Ho creato un nuovo file chiamato `giochi_EU_JP.csv`, aprendolo in modalità scrittura. Ho letto tutte le righe del file per poterlo popolare, usando la funzione `riempimento_fileA2`, passando indici di colonna diversi per ogni file. Una volta chiusi tutti i file, ho aperto solo quello nuovo in modalità lettura, per evitare eventuali modifiche.

Ho attuato la lettura della prima riga e ho creato due dizionari vuoti, uno per i giochi più venduti in Europa e uno per quelli in Giappone. Ho aperto un ciclo `while` in cui alla fine di questo viene letta la riga successiva, fino alla fine di queste. Al suo interno ho generato il popolamento dei dizionari solo

RELAZIONE DI TANIA DENTE

nel caso in cui la vendita superi una certa soglia di numero. Per chiave composta ho utilizzato il nome del gioco e il tipo di piattaforma e come valore la loro vendita.

Una volta riempiti, genero altri due dizionari vuoti in cui andrò a sommare i valori delle vendite di uno stesso gioco. Perciò, i due nuovi dizionari presenteranno come chiave il nome del gioco e come valore la somma delle vendite sulle diverse piattaforme.

Tramite l'operazione `max` ricavo la vendita più grande e la chiave alla quale è associata, per poter finalmente risolvere il primo punto della consegna.

Il quesito B3 chiede di risolvere due sottoquesiti. Per identificare il genere che ha venduto di più in Nord America ho popolato, tramite la funzione `riempimento_fileB3`, un nuovo file chiamato `genere_NA`.

Successivamente ho creato un dizionario che ho popolato usando come chiave il genere del gioco e come valore la somma delle vendite in Nord America dei giochi con lo stesso genere. Tramite l'operazione `max` ho rilevato il valore più alto e la chiave associata ad esso e così ho completato il primo sottoquesito.

Il secondo mi chiede lo sviluppatore che ha sviluppato più giochi. Essendo che in due file non è presente la categoria "sviluppatore", ho creato un altro file che ho popolato solo con le informazioni presenti nel terzo file, eliminando eventuali record superficiali.

Anche qui ho creato due dizionari, dove nel primo la chiave composta gioco-piattaforma determina il nome dello sviluppatore, e nel secondo la chiave diventa lo sviluppatore e il valore la somma delle volte in cui quello sviluppatore compare nel dizionario precedente. Determino il massimo tra i valori del dizionario `riepilogo` e arrivo alla risposta finale.

La domanda C1 è l'ultima della consegna. Anche in questo caso la risoluzione segue lo schema che ho utilizzato nei quesiti precedenti, generando dizionari intermedi per poi arrivare alla soluzione tramite l'operazione `max`.

L'unica differenza è nella logica per creare questi dizionari. Infatti, in `diz_annogenere` ho utilizzato l'operazione `append` per aggiungere i generi all'interno dei valori del dizionario che hanno come chiave uno specifico anno, in `diz_annogenere_sistematato` ho utilizzato l'operazione `set` per cancellare eventuali ripetizioni e infine in `diz_anno_quantità` ho determinato il numero di generi per ogni anno.

La complessità del mio codice è lineare perché i cicli che ho generato sono sequenziali e non annidati sulla stessa collezione di dati. Il punto su cui mi vorrei soffermare è la funzione `divisione` che, sebbene contenga dei cicli `for`, essi iterano sulla lunghezza della singola riga e non sul numero totale di righe del file. Poiché la lunghezza di una riga è considerata "piccola" e costante rispetto al numero di record, non trasforma l'algoritmo in quadratico.

In tutte le domande sono arrivata alla medesima conclusione. Dovendo leggere tutti i dati almeno una volta per poter rispondere ai quesiti correttamente, la complessità è lineare sia nel caso medio che nel caso pessimo.